

Messages, Callbacks, and the Inter-Agent Chat Log

Messages are the runtime substrate that makes the rest of the agent system legible. Tasks describe what an agent should do, but messages describe what actually happened: a request was routed, a callback arrived, a self-check was queued, or the hypervisor observed a line of chat that had not yet been processed. In this system, the message stream is not an incidental log; it is part of the manuscript’s operational memory.

The basic unit is a routed message between named agents. A message records a sender, a recipient, and a human-readable payload, and it is rendered in an append-only chat log in the form **from_agent --> to_agent: message**. That format is intentionally simple. It can be scanned by a human, parsed by tooling, and replayed as evidence of how a section was produced. Because the log is append-only, later work does not erase earlier coordination; it accumulates beside it.

Routed messages serve several distinct purposes. Some are direct work requests, such as asking a section agent to draft or revise prose. Others are coordination messages between sibling agents, for example when one section agent needs a neighboring section to align terminology or avoid duplication. Still others are callback messages that report the result of a completed action. The system treats these callbacks as first-class work items rather than as passive notifications.

A particularly important callback type is **process_message**. When an agent receives a routed message that requires interpretation, the runtime can enqueue a **process_message** task so the recipient handles the message explicitly. This makes message handling durable and inspectable. Instead of assuming that a message was read immediately and correctly, the system records the act of processing as a task with its own status, prompt, and outcome. In practice, this means that a callback can become the next concrete unit of work: revise a section from feedback, request research, coordinate with a sibling, or simply acknowledge receipt.

Self-messages are another useful pattern. An agent may send a message to itself when it needs to preserve a decision, stage a follow-up action, or convert a reflective note into queued work. For section agents, self-messages often support planning: after inspecting the outline, the section payload, registered references, and diagram opportunities, the agent can queue a self-addressed plan that becomes visible in the same durable chat stream as external coordination. This keeps internal deliberation from disappearing into hidden state.

The chat log is therefore both operational and readable. It shows the sequence of interactions that led to a draft, but it also exposes the system’s intermediate reasoning in a form that can be audited later. The hypervisor can inspect unprocessed chat lines directly, which gives it access to the live edge of coordination rather than only to completed task records. That access matters when the hypervisor needs to intervene, reconcile competing actions, or detect that a callback has arrived but has not yet been handled by the intended agent.

This separation between messages and tasks is deliberate. Tasks describe durable obligations with status transitions; messages describe the conversational surface through which those obligations are negotiated, delegated, and confirmed. The result is a runtime in which coordination is visible at two levels at once: the queue shows what must be done, and the chat log shows how the system talked itself into doing it.

The same distinction also clarifies why the system keeps message handling explicit. A routed message may be informative, directive, or corrective, but it does not by itself change the state of the manuscript. Only when the recipient processes it does the message become operationally meaningful. That extra step is what makes the chat log useful as a source of provenance: one can see not only that a request was issued, but also that it was acknowledged, interpreted, and converted into a concrete follow-up action.

In practice, this means the inter-agent chat log is not merely a transcript of conversation. It is a

coordination ledger. It records the handoff from one agent to another, the return path of callbacks, the internal notes that agents preserve for themselves, and the unprocessed lines that remain visible to the hypervisor. Taken together, those traces make the runtime inspectable without making it opaque to the agents that depend on it.

The section is intentionally self-contained and does not require a visual to carry its argument, so no diagram is requested here.